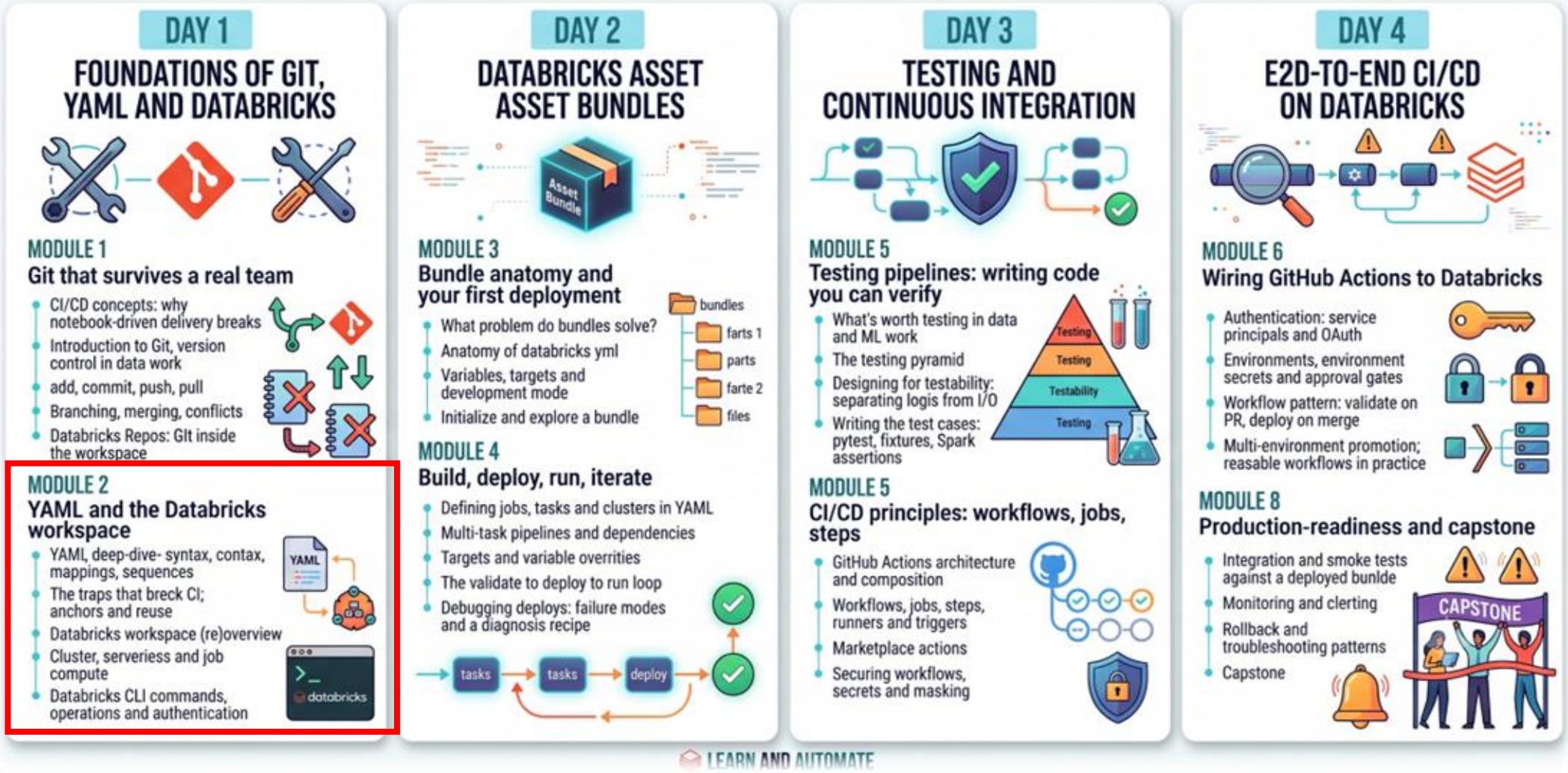
DataOps Mastery: Advanced CI/CD and Pipeline Automation

Module 2 – YAML and the Databricks Workspace

Asst. Prof. Dr. Santitham Prom-on

Department of Computer Engineering, Faculty of Engineering
King Mongkut's University of Technology Thonburi

Course Overview



BIG DATA EXPERIENCE

Module 2

YAML and the Databricks workspace

KMUTT

Module Overview

By the end of this module, students can

- Read any YAML file as nested mappings, sequences and scalars.
- Write correctly indented YAML from a set of requirements.
- Control how a value is typed, and know when quoting is necessary.
- Use multi-line strings, comments, anchors and aliases appropriately.
- Distinguish YAML syntax from a tool's schema, and check each separately.
- Configure and verify a Databricks CLI profile, and run a job from the terminal.

BIG DATA EXPERIENCE

Why YAML?

- It describes structure, not behavior
  - YAML holds configuration. It contains no logic and executes nothing. The application that reads the file decides what the keys mean.
- It is designed to be read and edited by people
  - No braces, no commas, no closing tags. That is the whole reason the industry settled on it for configuration rather than JSON or XML.
- It lives in Git, so configuration gets reviewed
  - Configuration as code
- The name, and the extension
  - YAML Ain't Markup Language.

KMUTT

BIG DATA
EXPERIENCE

YAML

Part 1

KMUTT

BIG DATA EXPERIENCE

A YAML document is a tree

The file is an encoding. The tree is the object. Every tool reads the tree.

WHAT YOU WRITE

```yaml
pipeline:
  name: customer_features
  enabled: true
  owners:
    - data-science
    - data-engineering
  settings:
    retries: 3
```

WHAT THE TOOL RECEIVES

```
{
  "pipeline": {
    "name": "customer_features",
    "enabled": true,
    "owners": ["data-science",
               "data-engineering"],
    "settings": {"retries": 3}
  }
}
```

KMUTT

BIG DATA EXPERIENCE

YAML vs Python vs JSON

YAML has no structure that Python and JSON do not already have

YAML term	Python	JSON	YAML example
Mapping	dict	object	pipeline, settings
Sequence	list	array	owners
Scalar	str, int, float, None	string, number, true/false, null	customer_features, true, 3

KMUTT

BIG DATA EXPERIENCE

The same object, in two notations

If you can write the dictionary, you can write the YAML

PYTHON

```
config = {
    "name": "churn-model",
    "workers": 4,
    "enabled": True,
    "tags": ["ml", "analytics"],
    "compute": {
        "runtime": "15.4"
    }
}
```

YAML

```
name: churn-model
workers: 4
enabled: true
tags:
  - ml
  - analytics
compute:
  runtime: "15.4"
```

Braces become indentation. Commas disappear. Quotes become optional. Nothing else changes — including which values need to stay strings.

KMUTT

BIG DATA EXPERIENCE

Scalars

A single value at a leaf of the tree

- **key: value**
  - One key, one scalar. This is the entire syntax at a leaf.
- **Four kinds**
  - Strings, numbers, booleans and null.
  - You write literal characters; the parser decides which of the four you meant.
- **Empty is not one thing**
  - tags: [] — a sequence (a list) with no elements. In Python, [].
  - parameters: {} — a mapping (a dictionary) with no keys. In Python, {}.
  - description: null — the null value, meaning there is no value. In Python, None. Writing ~, or writing nothing at all after the colon, gives the same result.
  - empty: "" — a string containing no characters. In Python, "".

KMUTT

Mappings

A value may itself be a set of key-value pairs

NESTING BY INDENTATION

```
compute:
  runtime: "15.4"
  workers: 2
  autoscale:
    min_workers: 2
    max_workers: 8
```

THE RULES

Spaces only. A tab is an error.

Sibling keys share a column. runtime, workers and autoscale are siblings because all three start at column 3.

Two spaces per level is the convention. Keys are case-sensitive.

Indentation is not formatting. It is the only thing that expresses which key belongs to which parent.

BIG DATA EXPERIENCE

Sequences

An ordered list of values

A LIST OF SCALARS

```
libraries:
  - pandas
  - scikit-learn
  - mlflow
```

WHAT IT BECOMES

```
{
  "libraries": [
      "pandas",
      "scikit-learn",
      "mlflow"
  ]
}
```

Every `- ` marker in one list must start in the same column. Misalign one and you get a different structure, usually without an error.

KMUTT

BIG DATA EXPERIENCE

Sequences of mappings

The single most common shape in this course

EACH ELEMENT IS A MAP

```
tasks:
  - name: prepare_data
    notebook: notebooks/prepare
    retries: 2
  - name: train_model
    notebook: notebooks/train
    retries: 1
```

HOW TO READ IT

`- ` opens a new element.

notebook and retries belong to the element above them because they are indented to the same column as the text after `- `.

Two elements here, each with three keys.

tasks in a bundle, steps in a workflow, and every pipeline definition you write this week has exactly this shape.

KMUTT

Reading a YAML file

Generic approach

THE FILE

```
resources:
  jobs:
    nightly:
      name: nightly-scoring
      tasks:
        - task_key: score
          libraries:
            - pypi:
                package: mlflow
```

THE METHOD

1 Find the leftmost column. Those keys are the top level: here, resources alone.
2 Follow one branch down, ignoring the rest.
3 At each line ask only: mapping, sequence, or scalar?
4 A `- ` means you have entered a list. Count how many you passed to know how deep you are.
5 Confirm with yq -o json before you edit.

BIG DATA EXPERIENCE

LAB

Challenge 1 — Parse YAML

6 min

1. Open fixtures/pipeline.yml. Do not run anything yet.
2. On paper, draw the tree. Mark every node as a mapping, a sequence, or a scalar.
3. Write down how many elements the tasks sequence has, and how many keys each element has.
4. Now check: `yq -o json . fixtures/pipeline.yml`

KMUTT

BIG DATA EXPERIENCE

```
santitham@data-engineer:~/m2-demo/assets$ yq -o json . fixtures/pipeline.yml
{
  "pipeline": {
    "name": "customer_features",
    "enabled": true,
    "owners": [
      "data-science",
      "data-engineering"
    ],
    "settings": {
      "retries": 3,
      "timeout_minutes": 20
    }
  },
  "tasks": [
    {
      "name": "prepare_data",
      "notebook": "notebooks/prepare",
      "retries": 2
    },
    {
      "name": "train_model",
      "notebook": "notebooks/train",
      "retries": 1
    }
  ]
}
```

KMUTT

BIG DATA EXPERIENCE

Writing YAML

Part 2

KMUTT

BIG DATA EXPERIENCE

Step 1 — A flat mapping

Start with the values that have no structure

WHAT YOU WRITE

```
name: churn-model
owner: data-science
version: "1.0"
enabled: true
retries: 3
```

NOTES

One key per line, a colon, a space, then the value. The space is required.

version is quoted because 1.0 would otherwise become the number 1.

enabled is a real boolean. retries is a real integer. Neither is quoted.

KMUTT

Step 2 — Group related keys

Nest them under a parent

WHAT YOU WRITE

```
project:
  name: churn-model
  owner: data-science
  version: "1.0"

notifications:
  enabled: true
  channel: "#data-alerts"
```

NOTES

project: has no value on its own line.
Its value is everything indented beneath it.

The blank line is decoration. It changes nothing about the structure.

"#data-alerts" must be quoted, or the # starts a comment and the value is null.

A key with an indented block beneath it has that block as its value. A key with nothing after it has null.

Step 3 — Add a list

For values that are ordered and repeatable

WHAT YOU WRITE

```
tags:
  - machine-learning
  - customer-analytics

environments:
  development:
    workers: 1
  production:
    workers: 4
```

NOTES

tags is a sequence: order matters, names do not.

environments is a mapping: the names development and production matter, and order does not.

Choosing between these two is a design decision, not a syntax one.

Step 4 — A list of mappings

When each item needs several fields

WHAT YOU WRITE

```
tasks:
  - name: prepare_features
    entrypoint: notebooks/prepare
    retries: 2

  - name: train_model
    entrypoint: notebooks/train
    retries: 1
```

NOTES

The `- ` and the first key share a line. Everything after it aligns to the column where that first key begins.

Get that column wrong by one space and the parser builds a different tree — sometimes without complaining.

If you write only one shape correctly from memory, make it this one.

BIG DATA EXPERIENCE

Mapping or sequence?

A design decision, and the one beginners get wrong

USE A MAPPING WHEN

Each item has a name that means something, and you will refer to it by that name.

```
environments:
  development:
    workers: 1
  production:
    workers: 4
```

Order is irrelevant. Names are the point.

USE A SEQUENCE WHEN

Items are alike, order may matter, and nothing refers to one by name.

```
tasks:
  - name: prepare
    retries: 2
  - name: train
    retries: 1
```

You can add a third without renaming anything.

KMUTT

BIG DATA EXPERIENCE

Comments

The one part of the file the parser throws away

- Everything after # on a line is discarded
- Record the why, not the what
- A # inside a value needs quoting "#data-science"

KMUTT

BIG DATA EXPERIENCE

Convention

- Two spaces per level, never tabs
  - Configure your editor once: expand tabs, two-space indent, show whitespace.
- Comment the why, not the what
  - retries: 3 # transient S3 failures during peak hours.
- Quote anything that is not a quantity or a true Boolean
  - Versions, identifiers, dates, country codes, channel names, anything with a colon.
- Keep one document per file, and name it after what it configures

KMUTT

BIG DATA EXPERIENCE

Changing a file you did not write

- Parse it before you touch it
  - yq . on the original. If it already fails, that is not your fault and you need to know before you add a second problem.
- Change one value, never the indentation
  - Editing a value is almost always safe. Re-indenting a block is where structure gets silently rewritten.
- Copy the shape of the line above
  - To add a list element, duplicate an existing one and edit the copy. You inherit its indentation, which is the part that is easy to get wrong.
- Parse it again, and read the diff

KMUTT

BIG DATA EXPERIENCE

Types are resolved, not declared

- There is no type annotation in YAML
  - You cannot write that version is a string. Nothing in the syntax lets you say so.
- The parser pattern-matches the token
  - An unquoted value is compared against a table of patterns. The first match wins, and the decision is not reported anywhere.
- Quoting is how you take the decision back
  - "1.0" is a string because you said so. 1.0 is a float because the parser said so.

KMUTT

BIG DATA EXPERIENCE

Implicit type resolution

What an unquoted value becomes

Written	Resolves to	Type
42	42	integer
1.0	1.0	float
3.10	3.1	float — the zero is lost
true, True, TRUE	true	boolean
null, ~, or nothing	null	null
2026-08-16	a date object	timestamp
"3.10"	**3.10**	**string**

KMUTT

BIG DATA EXPERIENCE

When to quote

A short rule that covers almost every case

QUOTE IT

```
version: "1.0"
python_version: "3.10"
date_label: "2026-08-16"
country: "NO"
channel: "#data-alerts"
message: "key: value"
expression: "${var.catalog}"
```

LEAVE IT UNQUOTED

```
workers: 4
threshold: 0.85
enabled: true
description: null
name: churn-model
entrypoint: notebooks/train
```

If the value is a quantity or a true boolean, leave it. Otherwise quote it. The cost of quoting something unnecessarily is zero.

KMUTT

BIG DATA EXPERIENCE

Expressions are just strings

To YAML, at least

- Other systems embed their own syntax
  - ${var.catalog} in a bundle, ${{ github.sha }} in a workflow. YAML has no idea what either of these means.
- YAML's only job is to deliver the string intact
  - The consuming tool substitutes the value afterwards, long after parsing is over.
- Which is why they need quoting
  - A value beginning with { is flow-mapping syntax to YAML. Quote it and the problem disappears entirely.

KMUTT

LAB

Challenge 2 — Predict the type

7 min

1. Write down the resolved type of each: 42 · 1.0 · 3.10 · 0755 · true · null · 1e3 · 2026-08-16 · "3.10"
2. Commit to all nine before running anything. A guess you did not write down does not count.
3. Check each with: echo 'k: <value>' > t.yml && yq '.k | type' t.yml
4. For any you got wrong, find the row of the table that explains it.
5. Then the question that matters: which of the nine would you have quoted, before today?

The same lines under each indicator

What the consumer actually receives

YOU RUN

```
literal: |
  line one
  line two

folded: >
  line one
  line two

literal_strip: |-
  line one
  line two

plain_multiline: line one
  line two

$ yq -o json . fixtures/scalars.yml
```

YOU SEE

```
{
  "literal":
      "line one\nline two\n",

  "folded":
      "line one line two\n",

  "literal_strip":
      "line one\nline two",

  "plain_multiline":
      "line one line two"
}

A plain multi-line scalar folds,
exactly like >.
```

A run: block under both indicators

The same two commands, and the two strings the shell would receive

```
$ cat fixtures/runblock.yml
$ yq -o json . fixtures/runblock.yml
```

```
santitham@data-engineer:~/m2-demo/assets$ cat fixtures/runblock.yml
steps:
  - name: literal
    run: |
      pip install flake8
      flake8 etl/ --max-line-length=100

  - name: folded
    run: >
      pip install flake8
      flake8 etl/ --max-line-length=100
santitham@data-engineer:~/m2-demo/assets$ yq -o json . fixtures/runblock.yml
{
  "steps": [
    {
      "name": "literal",
      "run": "pip install flake8\nflake8 etl/ --max-line-length=100\n"
    },
    {
      "name": "folded",
      "run": "pip install flake8 flake8 etl/ --max-line-length=100\n"
    }
  ]
}
```

BIG DATA EXPERIENCE

Anchors and aliases

Define once, expand many times

SOURCE

```
default_compute: &default_compute
  workers: 2
  runtime: "15.4"

development:
  compute: *default_compute
production:
  compute: *default_compute
```

AFTER PARSING

```
development:
  compute:
    workers: 2
    runtime: "15.4"
production:
  compute:
    workers: 2
    runtime: "15.4"
```

Expansion happens in the parser. The consumer receives a fully expanded tree and cannot tell an anchor was used.

KMUTT

Use them sparingly

- They give exact copies, not variants
- They do not cross files
- Overuse makes a file harder to read than the repetition did
- Tool-native reuse is usually better

BIG DATA EXPERIENCE

Three layers of correctness

- 1 - YAML syntax
  - Can a parser turn these characters into a tree? If not, nothing matters and the parser will tell you the line.
- 2 - The tool's schema
  - Are the expected keys present, in the expected places, holding the expected kinds of value? A YAML parser has no opinion about this.
- 3 - Runtime behavior
  - Do the referenced notebooks, credentials, clusters and commands actually exist and work? Only running it answers this.

KMUTT

BIG DATA EXPERIENCE

Checking each layer

The instruments, and what each is blind to

Layer	Command	Catches	Blind to
1 · Syntax	yq .	Tabs, bad indent, missing space	Anything that parses
1 · Types	yq '<path> | type'	3.10 became a float	Anything you don't ask about
2 · Style	yamllint	Duplicate keys, truthy values	Tool-specific structure
2 · Schema	actionlint	Unknown keys, wrong inputs	Whether the file exists
3 · Runtime	run it	Everything else	Nothing — but it is slow

Work down the list. Each check is cheaper and more precise than the one below it, and each is blind to what the next one sees.

KMUTT

BIG DATA EXPERIENCE

yq

The instrument you will use most

- **yq . file.yml**
  - Parse and print the tree back. A failure means the file is not valid YAML, and the message carries a line and a column.
- **yq -o json . file.yml**
  - The same tree as JSON. The clearest way to see what a tool receives.
- **yq '<path> | type' file.yml**
  - Report the resolved type of one value: !!str, !!int, !!float, !!bool.

KMUTT

yq on a valid file

Parsing, and the same document as JSON

```
santitham@data-engineer:~/m2-demo/assets$ yq . fixtures/pipeline.yml
pipeline:
  name: customer_features
  enabled: true
  owners:
    - data-science
    - data-engineering
  settings:
    retries: 3
    timeout_minutes: 20
tasks:
  - name: prepare_data
    notebook: notebooks/prepare
    retries: 2
  - name: train_model
    notebook: notebooks/train
    retries: 1
```

```
santitham@data-engineer:~/m2-demo/assets$ yq -o json . fixtures/pipeline.yml
{
  "pipeline": {
    "name": "customer_features",
    "enabled": true,
    "owners": [
      "data-science",
      "data-engineering"
    ],
    "settings": {
      "retries": 3,
      "timeout_minutes": 20
    }
  },
  "tasks": [
    {
      "name": "prepare_data",
      "notebook": "notebooks/prepare",
      "retries": 2
    },
    {
      "name": "train_model",
      "notebook": "notebooks/train",
      "retries": 1
    }
  ]
}
```

BIG DATA EXPERIENCE

Six mistakes worth recognising

Ordered by how hard they are to notice, not how often they happen

Written	What you get	Caught by
A tab in the indentation	Nothing — the file does not parse	yq, immediately
name:training-job	Parse error, or a scalar	yq, immediately
environment: twice	One value silently discarded	yamllint
A misaligned - marker	One list element instead of two	nothing, usually
python-version: 3.10	The number 3.1	nothing — you must look
A key indented one level too deep	A correct value in the wrong place	the tool's schema

The first two stop you. The last four do not, which is why the type check and the schema check are worth running rather than trusting your reading.

KMUTT

The two that parse cleanly

Same file, two parsers, three different answers

WHAT IS IN THE FILE

```
on:
  push:
    branches: [main]
python-version: 3.10
enabled: no
country: NO
```

WHAT EACH PARSER RETURNS

	yq v4 (1.2)	PyYAML (1.1)
on:	"on"	true
python-version	3.1	3.1
enabled	"no"	false
country	"NO"	false

YAML 1.1 read yes/no/on/off as booleans; 1.2 removed that. Which specification applies depends on the parser reading your file, not on the file.

BIG DATA EXPERIENCE

One file, two parsers

The demonstration, run on your own machine

```
$ yq -o json . fixtures/two-parsers.yml
$ python3 -c "import yaml,json;print(json.dumps(yaml.safe_load(open('fixtures/two-parsers.yml')),indent=2))"
```

```
santitham@data-engineer:~/m2-demo/assets$ yq -o json . fixtures/two-parsers.yml
{
  "on": {
    "push": {
      "branches": [
        "main"
      ]
    }
  },
  "python-version": 3.10,
  "enabled": "no",
  "country": "NO"
}
santitham@data-engineer:~/m2-demo/assets$ python3 -c "import yaml,json;print(json.dumps(yaml.safe_load(open('fixtures/two-parsers.yml')),indent=2))"
{
  "true": {
    "push": {
      "branches": [
        "main"
      ]
    }
  },
  "python-version": 3.1,
  "enabled": false,
  "country": false
}
```

Reading a parser error

- found character that cannot start any token
- mapping values are not allowed in this context
- did not find expected key
- The reported line is where it noticed, not where you erred

LAB

Challenge 3 — Break it, then read the message

10 min

1. Copy fixtures/pipeline.yml to work.yml. Restore it to this state before each step.
2. Replace the leading spaces on one line with a tab. Run yq and read the message.
3. Remove the space after a colon. Predict the message before you run it.
4. Indent one `- ` marker by one extra space. Run yq — then explain why there is no error.
5. Duplicate a key with a different value. Find the one tool that objects.

BIG DATA EXPERIENCE

LAB

Challenge 4 — Author a configuration from requirements

12 min

1. Start from assets/skeleton.yml — five empty keys and nothing else.
2. Add a project name and owner; development and production environments with different worker counts.
3. Add two tasks, each with a name, entrypoint and retry count; a boolean controlling notifications.
4. Add a multi-line description with line breaks preserved; two tags; a version whose exact text must survive.
5. Validate with yq, then swap with your neighbour and have them state your structure aloud.

Success: yq -o json shows the tree you intended, and your neighbour read it the same way you did.

KMUTT

BIG DATA EXPERIENCE

The same structures, in the files you meet next

Read these as trees. The schemas are Modules 3 and 6; the shapes are today.

DATABRICKS ASSET BUNDLE

```
bundle:
  name: customer-churn
resources:
  jobs:
    training_job:
      name: training-job
      tasks:
        - task_key: train
          notebook_task:
            notebook_path: ./src/train
```

GITHUB ACTIONS WORKFLOW

```
name: Validate configuration
on:
  pull_request:
jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - name: Run validation
        run: example-command validate
```

KMUTT

BIG DATA EXPERIENCE

databricks/

databricks-cli

A command line interface tool to automate your databricks platform.

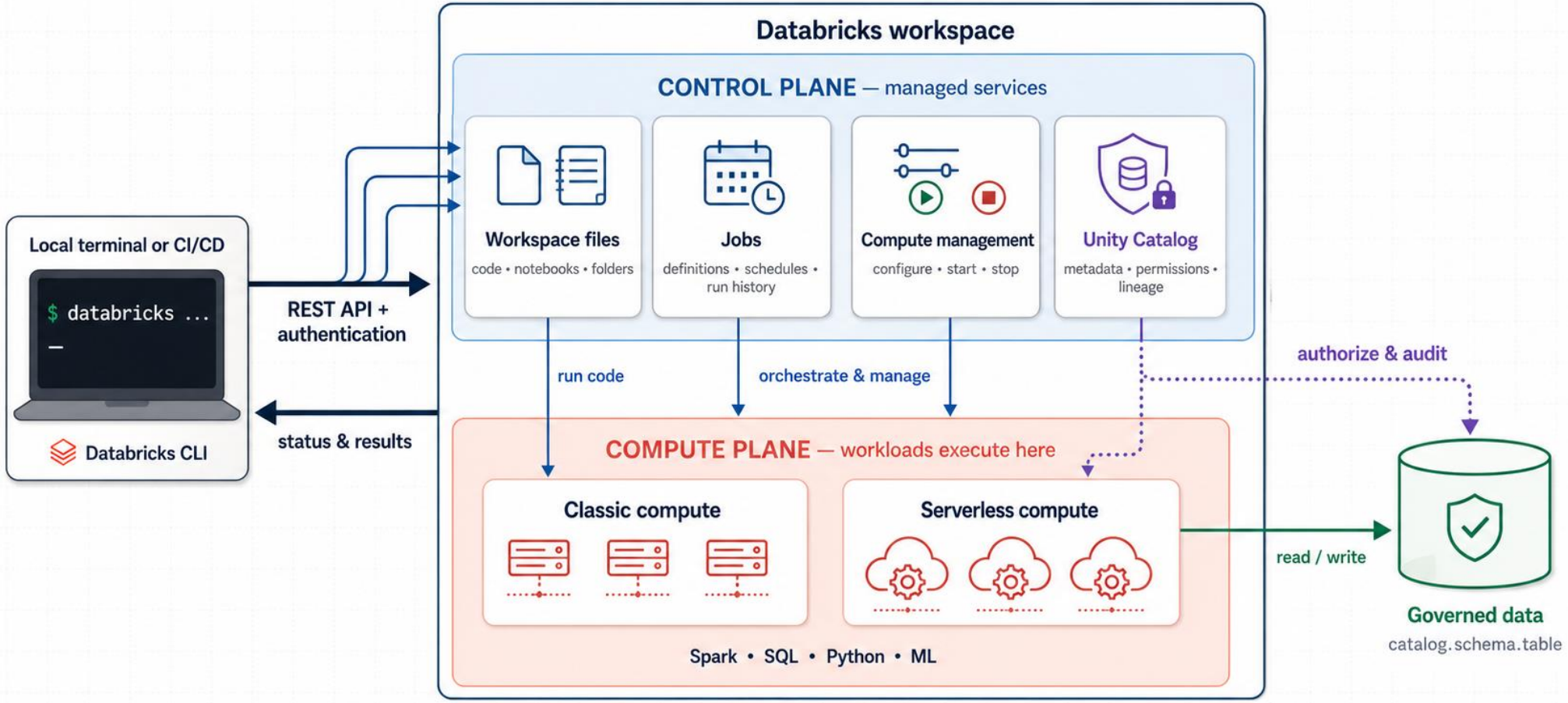
The workspace and the CLI

Part 3

KMUTT

Databricks Workspace + CLI

Control in the control plane. Execute in the compute plane.



BIG DATA EXPERIENCE

Compute types

Three kinds, and they are not interchangeable

	All-purpose	Jobs compute	Serverless
Created by	A person, once	The job, per run	The platform
Lifetime	Until stopped	The run only	The run only
Configuration	Edited in the UI	Declared in a file	Mostly fixed
Start-up	Already warm	Several minutes	Seconds
Cost profile	Highest	Lower	Per query

KMUTT

BIG DATA EXPERIENCE

Which compute for automation

Why a jobs cluster, and not the one you already have running

- **An all-purpose cluster is shared mutable state**
  Someone installs a library, changes a Spark setting, or restarts it on a different runtime. None of that is recorded in your repository.
- **Which makes runs irreproducible**
  A pipeline that passes today can fail tomorrow for a reason that appears in no commit and no diff.
- **Jobs compute is declared, not maintained**
  The specification lives in a file, under review, in Git. That is the property automation needs, and it is the reason to prefer it.
- **Serverless trades control for latency**
  No start-up wait, less control over the runtime. Reasonable for short tasks.

KMUTT

The CLI and the workspace

A local program issuing remote calls

- **The same relation Git has to GitHub**
  From Module 1: some operations are local, some cross the network. Confusing the two causes most beginner errors.
- **Except that almost everything here is remote**
  The CLI holds no local state beyond a configuration file. Every command this afternoon is an API call.
- **Two programs share the name**
  The modern CLI is one Go binary, v0.205 or later. The legacy databricks-cli is a Python package with incompatible subcommands.

`databricks --version` distinguishes them: `Databricks CLI v0.2xx.x` against `databricks-cli, version 0.18.0`.

The command surface

Six groups cover everything this course does

Group	What it operates on	Used in
databricks auth / configure	Profiles and credentials	Today
databricks workspace	Files and folders in the workspace	Today
databricks jobs	Job definitions and runs	Today, and Module 8
databricks bundle	Deploy a whole project from a file	Modules 3, 4 and 7
databricks clusters	Compute lifecycle	Rarely, once bundles exist
databricks fs	DBFS paths	Occasionally

Every group takes --profile and --output json. Learn those two flags and the rest of the CLI is discoverable with --help.

BIG DATA EXPERIENCE

Version check

Confirming which of the two programs you have

```
$ databricks --version
```

```
santitham@data-engineer:~/m2-demo/assets$ databricks --version
Databricks CLI v1.9.0
```

KMUTT

Authentication

A host, and a credential

- **Personal access token**
  Generated from User Settings → Developer → Access tokens.
  Set a short expiry: a token with no expiry outlives the course.
- **It is a bearer credential**
  Anyone holding it acts as you, with your permissions. There is no second factor and no confirmation step.
- **Where it must never go**
  Not committed. Not pasted into a notebook. Not written into a workflow file. Not sent in a chat message.
- **This is temporary**
  Module 7 replaces it with a service principal, which is what a pipeline should authenticate as.

The configuration file

One file, several named profiles

~/.databrickscfg

```
[dev]
host  = https://adb-1234.5.azuredatabricks.net
token = dapi**************************

[staging]
host  = https://adb-6789.0.azuredatabricks.net
token = dapi**************************
```

SELECTING ONE

databricks <cmd> --profile dev

export DATABRICKS_CONFIG_PROFILE=dev

The dev / staging / prod split you create here becomes bundle targets in Module 4, and deployment environments in Module 7.

The file lives outside any repository by design. That, not .gitignore, is what keeps the token out of your history.

configure, and the file it writes

Two commands and their result

```
$ databricks configure --host https://<your-workspace> --profile dev
$ cat ~/.databrickscfg
```

Workspace: https://adb-1502583690645883.3.azuredatabricks.net

```
santitham@data-engineer:~$ cat .databrickscfg
; The profile defined in the DEFAULT section is to be used as a fallback when no profile is explicitly specified.
[DEFAULT]

[dev]
host  = https://adb-1502583690645883.3.azuredatabricks.net
token = [illegible]

[__settings__]
default_profile = dev
```

BIG DATA EXPERIENCE

Verify before use

databricks current-user me

- **This is the schema check for the CLI**
  It confirms three things at once: the configuration parsed, the host is reachable, and the credential was accepted.
- **Run it before any real work**
  It converts a class of confusing downstream failures into one clear failure, at the moment you know what you just changed.
- **Read the failures deliberately**
  A wrong host, an expired token and a missing profile each produce a different message. Recognising them is the skill.

Same discipline as running yq before you rely on a file: check the cheap thing first, where the answer is unambiguous.

KMUTT

LAB

Challenge 5 — Configure and verify

8 min

1. Generate a personal access token with a short expiry. Note the expiry date now.
2. Run: databricks configure --host <your-workspace> --profile dev
3. cat ~/.databrickscfg and confirm the profile name and host are what you expect.
4. Verify: databricks current-user me --profile dev | jq .userName
5. Now break it deliberately: change the host to a wrong value, re-run, and read the error.

Success: a working dev profile, and you recognise the error a wrong host produces.

Reading the output with jq

Every command can return JSON; jq extracts one field

YOU RUN

```
# human-readable by default
$ databricks workspace list "$WS" \
    --profile dev

# machine-readable on request
$ databricks workspace list "$WS" \
    --profile dev --output json

# one field out of the structure
$ databricks workspace list "$WS" \
    --profile dev --output json \
  | jq -r '.[].path'
```

YOU SEE

```
hello_world

[
  {
    "object_type": "NOTEBOOK",
    "path": "/Workspace/.../hello_world",
    "language": "PYTHON"
  }
]

/Workspace/.../hello_world
```

```
santitham@data-engineer:~$ databricks workspace list / --profile dev
ID                  Type        Language  Path
1950788369110222    DIRECTORY               /Repos
1950788369110219    DIRECTORY               /Shared
1950788369110218    DIRECTORY               /Users
```

BIG DATA EXPERIENCE

Three jq patterns cover this course

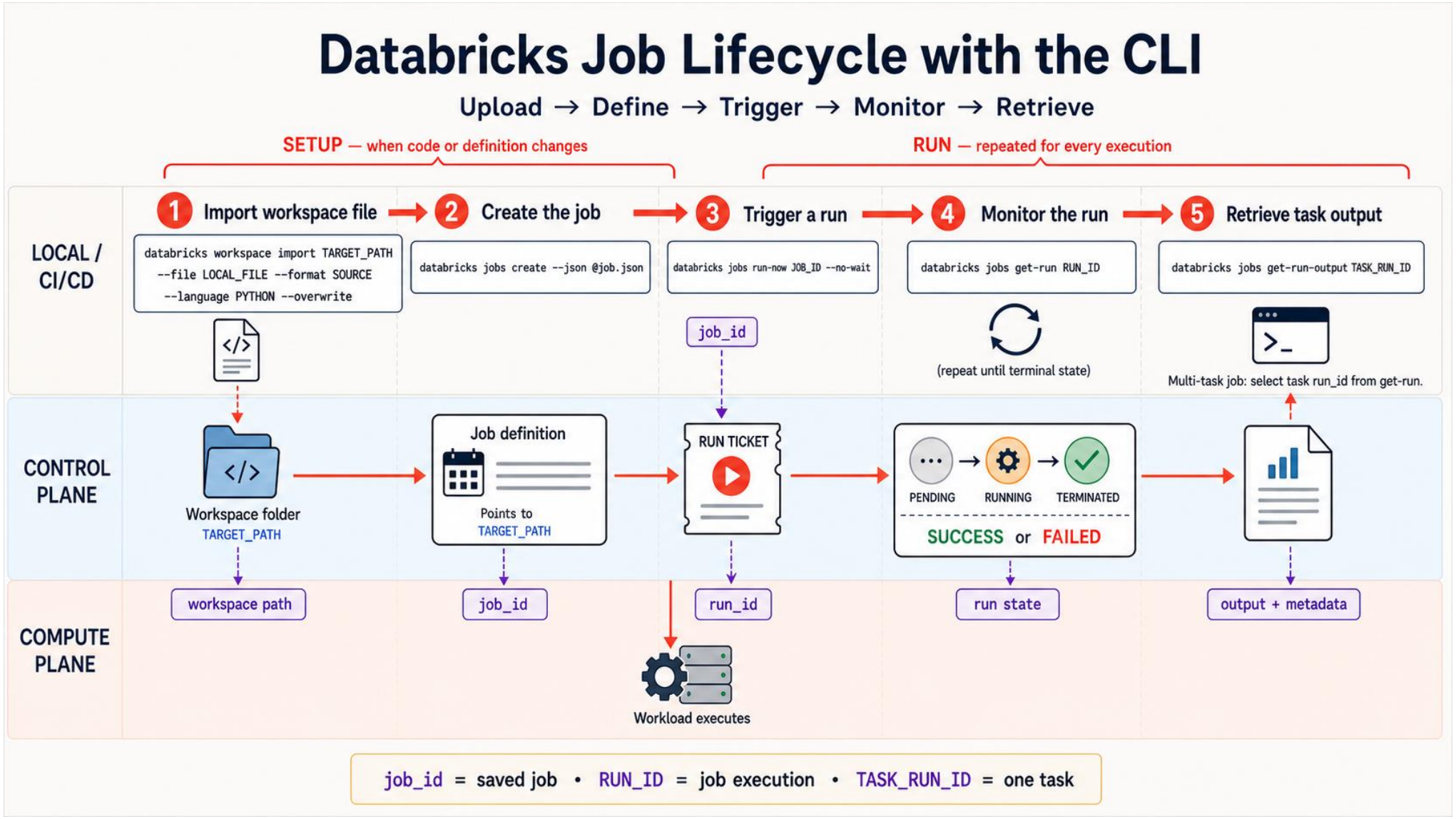
You do not need to learn jq properly

- **jq .**
  Pretty-print the whole structure. Run this first, always, to see what you are working with before writing a path.

- **jq -r '.field.subfield'**
  Extract one value. -r strips the surrounding quotes, which matters when you assign the result to a shell variable.

- **jq -r '.[].name'**
  Extract one field from every element of a list.

KMUTT


Databricks Job Lifecycle with the CLI
Upload → Define → Trigger → Monitor → Retrieve
SETUP — when code or definition changes
RUN — repeated for every execution
LOCAL / CI/CD
CONTROL PLANE
COMPUTE PLANE
1 Import workspace file
2 Create the job
3 Trigger a run
4 Monitor the run
5 Retrieve task output
databricks workspace import TARGET_PATH --file LOCAL_FILE --format SOURCE --language PYTHON --overwrite
databricks jobs create --json @job.json
databricks jobs run-now JOB_ID --no-wait
databricks jobs get-run RUN_ID
databricks jobs get-run-output TASK_RUN_ID
job_id
(repeat until terminal state)
Multi-task job: select task run_id from get-run.
Workspace folder
TARGET_PATH
Job definition
Points to
TARGET_PATH
RUN TICKET
PENDING
RUNNING
TERMINATED
SUCCESS or FAILED
workspace path
job_id
run_id
run state
output + metadata
Workload executes
job_id = saved job • RUN_ID = job execution • TASK_RUN_ID = one task

LAB

Challenge 6 — Full lifecycle without the UI

20 min

1. Following the instruction files (lab.md)
2. Create the job
3. Run
4. Get the result

Success: a SUCCESS result state obtained by command, and jq used to extract at least three fields.

The End of Module 2

If you have any further questions, please contact me via santitham.pro@kmutt.ac.th